

Verifiable Agentic Optimization: Trajectory Diagnostics for Targeted Post-Training

Anonymous Authors
Anonymous Institution
anonymous@example.com

Abstract

Modern language-model agents are increasingly evaluated on tasks in which they iteratively modify a structured artifact, observe an external verifier, and continue under a fixed budget. Standard endpoint metrics—pass rate, best score, or time-to-threshold—hide why an agent succeeds or fails. We introduce *verifiable agentic optimization* (VAO), a task class in which the primary object of study is not only the final artifact but the verified trajectory induced by the agent. We formalize VAO as an optimization process over editable artifacts, intervention modes, verifier outputs, and budgets. We then propose a diagnostic decomposition separating cost per step, routing mismatch, feedback use, and within-mode competence. The key methodological device is a mode-complete counterfactual step: at each checkpoint, the agent emits a distribution over editing modes and one candidate edit per mode; all candidates are verified offline, while only a controlled subset is exposed to the next online state. This produces a verified step-by-mode tensor from which routing regret, productive-mode proxies, feedback-alignment gains, and mode-conditioned competence estimates can be computed. We instantiate the framework for open-ended optimization of a stateful query engine implemented in `solution.py`. Finally, we specify a pre-registered experimental protocol and a targeted post-training pipeline in which an open-weight code model is first fine-tuned on routing records derived from verifier-generated labels. The contribution is a bridge between benchmark evaluation and diagnostic post-training: a task is not merely a scoreboard, but a source of causal hypotheses about how to improve weak backbones.

1 Introduction

Language-model agents are no longer evaluated only as static text generators. In software engineering, theorem proving, data processing, and scientific computation, an agent is often asked to inspect an environment, change a structured artifact, run a verifier, interpret the result, and continue until a time, token, or compute budget is exhausted. The unit of behavior in such settings is not a single answer. It is a trajectory: a sequence of hypotheses, edits, verifier outputs, budget expenditures, and revisions.

Yet the dominant evaluation interface remains an endpoint interface. A model passes or fails. A score improves or does not. A benchmark leaderboard ranks systems by best final performance. These statistics are necessary, but they are coarse. Two backbones can obtain similar final scores for different reasons: one may have a good prior over the task but weak use of verifier feedback; another may initially route to poor editing strategies but recover after informative errors; a third may select the right high-level mode but implement the local patch badly. Endpoint metrics compress these mechanisms into the same scalar.

This compression is especially harmful when evaluation is meant to inform post-training. If a weak backbone underperforms a strong one, what should be trained? Should the model learn to choose the right kind of intervention? Should it learn to interpret verifier diagnostics? Should it learn to produce better code once the right intervention mode is fixed? Or is the bottleneck simply cost per step? A benchmark that cannot distinguish these cases is a poor guide for targeted improvement.

We study a class of tasks we call *verifiable agentic optimization*. A task instance consists of an input, an editable artifact, an external verifier, a budget vector, and a finite or structured space of intervention modes. The agent iteratively proposes modifications. The verifier returns correctness, a scalar quality measure, and diagnostic information. The scientific object is the verified trajectory induced by the policy, not only the endpoint.

Our concrete instantiation is an open-ended code-editing benchmark for a stateful query engine. The artifact is a Python file, `solution.py`, containing a `CandidateQueryEngine` implementation. The verifier checks exact functional equivalence against a trusted reference implementation over generated workload traces and measures latency and memory relative to a baseline. The agent must improve the implementation under a fixed budget. We use six semantically meaningful editing modes: `layout`, `indexing`, `topk`, `caching`, `summaries`, and `micro`. At each step the agent emits a probability distribution over these modes and one concrete edit per mode. All six edits are verified offline. Initially, only the result of the top-probability edit is shown to the agent at the next step, preserving a

closed-loop online experience while producing counterfactual evidence for diagnostics.

This design yields a richer object than a normal trajectory. Each step produces a verified panel over modes. Across steps it produces a tensor indexed by run, step, mode, edit, and verifier output. From this tensor we estimate: (i) *routing mismatch*, the gap between the model’s distribution over modes and the modes that were actually productive; (ii) *feedback use*, the improvement in mode alignment after observing verifier evidence; (iii) *within-mode competence*, the quality of edits when the mode is held fixed; and (iv) *cost per step*. These quantities support a falsifiable post-training rule: if the dominant deficit is routing mismatch, train on routing records; if it is feedback use, train on verifier-summary-to-next-action records; if it is within-mode competence, train on mode-conditioned patch records.

Research questions. We formulate the work around five questions.

- RQ1: Can a verifiable iterative code-editing task be formalized so that trajectories, not only endpoints, are comparable across backbones?
- RQ2: Can offline verification of one edit per intervention mode produce reliable estimators for routing mismatch and productive-mode structure?
- RQ3: Can within-mode competence be separated from routing by using mode-conditioned runtime traces and shared-checkpoint controlled evaluations?
- RQ4: Does feedback visibility change the model’s future distribution over modes in a measurable and directionally correct way?
- RQ5: Can the resulting diagnostics select a targeted post-training intervention that improves an open-weight weak backbone more than undiagnosed generic fine-tuning?

Contributions. The paper makes four contributions.

- We define *verifiable agentic optimization*, a task-first formalism for agents that iteratively modify artifacts under verifier feedback and budget constraints.
- We introduce a mode-complete counterfactual protocol producing a verified step-by-mode tensor while preserving a closed-loop online trajectory.
- We derive operational estimators for routing mismatch, productive-mode proxies, feedback alignment, two forms of within-mode competence, and cost-normalized performance gaps.
- We specify a targeted post-training program in which verifier-derived decision records train an open-weight code model first on routing, with feedback-use and mode-conditioned patching as subsequent phases.

Paper organization. Section 2 situates the work among agent evaluation, code benchmarks, iterative refinement, and post-training. Section 3 defines the VAO task, trajectories, modes, and visibility regimes. Section 4 presents the diagnostic decomposition, estimators, and theoretical accounting results. Section 5 gives a complete experimental design without reporting results. Sections 6 and 7 discuss anticipated interpretations, failure modes, and staged future work.

2 Related Work

LLM agents and interactive decision making. ReAct interleaves reasoning and action, showing that language models can improve decision making by acting in an environment and updating plans from observations [Yao et al., 2023b]. Reflexion studies linguistic memory as a way for agents to learn from feedback without weight updates [Shinn et al., 2023]. Self-Refine frames iterative generation as feedback and refinement [Madaan et al., 2023]. Tree of Thoughts explores multiple reasoning branches before selection [Yao et al., 2023a]. Voyager demonstrates open-ended skill acquisition through executable code, environment feedback, and a growing skill library [Wang et al., 2023]. Our work differs by treating the trajectory as a source of quantitative diagnostic estimators and post-training records, not only as an inference-time control pattern.

Agent benchmarks. AgentBench evaluates LLMs across several interactive environments and emphasizes long-horizon reasoning and instruction following [Liu et al., 2024]. WebArena provides realistic self-hostable web environments for autonomous agents [Zhou et al., 2024]. These benchmarks measure agent behavior in rich environments, but generally report aggregate success metrics. VAO instead asks what a task must log and verify to identify distinct failure mechanisms.

Code generation and software-engineering benchmarks. HumanEval and Codex established functional correctness as a central metric for code generation [Chen et al., 2021]; APPS broadened code evaluation to programming-challenge settings [Hendrycks et al., 2021]; AlphaCode scaled sampling and filtering for competitive programming [Li et al., 2022]. SWE-bench evaluates models on real GitHub issues requiring repository-level code edits [Jimenez et al., 2024], and SWE-agent studies interfaces that allow language agents to edit codebases and run tests [Yang et al., 2024]. Our task is smaller and more controlled than SWE-bench by design. The goal is not only to solve realistic issues, but to instrument an editable artifact deeply enough that each step reveals routing, feedback use, and local competence.

Tool use and verifier-mediated learning. Toolformer trains language models to decide when and how to call external APIs [Schick et al., 2023]. Verifier-mediated programming benchmarks use tests or compilers to filter samples, while agentic coding systems use test output to guide subsequent edits. VAO adopts the verifier as a primitive and distinguishes three roles: endpoint evaluation, counterfactual diagnosis, and supervision generation.

Post-training and preference optimization. Instruction tuning and RLHF align models to user intent using demonstrations and preference signals [Ouyang et al., 2022]. Direct Preference Optimization simplifies preference learning without explicit reward-model training [Rafailov et al., 2023]. LoRA and QLoRA make fine-tuning large models feasible with parameter-efficient adapters [Hu et al., 2021, Dettmers et al., 2023]. Our post-training proposal is narrower: rather than generic imitation of a strong agent’s full trajectory, we construct targeted records for routing, feedback use, and within-mode patching, each tied to a diagnostic failure mode.

3 Problem Formulation

Definition 1 (Verifiable agentic optimization task). *A verifiable agentic optimization task is a tuple*

$$T = (\mathcal{X}, \mathcal{A}, \mathcal{V}, \mathcal{B}, \mathcal{M}),$$

where $\mathcal{X}$ is an instance space, $\mathcal{A}$ is a space of editable artifacts, $\mathcal{V}$ is an external verifier, $\mathcal{B}$ is a budget space, and $\mathcal{M}$ is a set of intervention modes. A task instance consists of $x \in \mathcal{X}$, an initial artifact $a_0 \in \mathcal{A}$, and an initial budget $b_0 \in \mathcal{B}$.

In our instantiation, x is a workload profile specifying trace families, seeds, trace lengths, repetition counts, key-space size, initialization size, and warmup policy. The artifact a_t is the source of `solution.py`. The verifier $\mathcal{V}$ loads the candidate implementation, checks exact correctness against a reference engine, measures latency and memory, and returns a scalar loss. The budget includes maximum steps, wall-clock time, context budget, verifier calls, token cost, and model-call timeout.

Definition 2 (Verifier output). *For an artifact a , the verifier returns*

$$\mathcal{V}(x, a) = (c(x, a), \mathcal{L}(x, a), d(x, a)),$$

where $c \in \{0, 1\}$ is correctness, $\mathcal{L} \in \mathbb{R} \cup \{\infty\}$ is a scalar loss to minimize, and d is diagnostic evidence such as failed operation, first divergence, per-family losses, timing traces, or memory measurements.

Definition 3 (Intervention modes). *The primary mode set for the query-engine task is*

$$\mathcal{M} = \{\text{layout}, \text{indexing}, \text{topk}, \text{caching}, \text{summaries}, \text{micro}\}.$$

Each edit has exactly one primary mode and may optionally have secondary modes. Main analyses use primary modes; secondary modes are used for robustness.

The modes correspond to distinct semantic hypotheses. `layout` changes the primary storage representation; `indexing` changes range access paths such as sorted-key/bisect logic; `topk` changes the selection algorithm for top- k queries; `caching` adds or changes memoization; `summaries` introduces precomputed aggregation structures such as prefix sums, Fenwick trees, or bucket summaries; `micro` performs constant-factor Python-level optimizations.

Definition 4 (Agent state). *At step t , the agent observes an information state*

$$h_t = (x, a_t, \mathcal{H}_{<t}^{\text{vis}}, b_t, \text{mem}_t),$$

where $\mathcal{H}_{<t}^{\text{vis}}$ is the verifier history visible to the agent, b_t is residual budget, and mem_t is an optional structured memory. In the first experimental phase, $\mathcal{H}_{<t}^{\text{vis}}$ contains the full accessible online history; compressed or Markovian state variants are ablations.

Definition 5 (Mode-complete policy step). A backbone M with policy π_M maps h_t to

$$\pi_M(h_t) = (q_t^M, \{e_t^M(m)\}_{m \in \mathcal{M}}),$$

where $q_t^M \in \Delta(\mathcal{M})$ is an explicit probability distribution over modes, and $e_t^M(m)$ is one candidate edit constrained to primary mode m .

Let $U(a, e)$ denote applying edit e to artifact a . The mode-complete counterfactual panel at step t is

$$\mathcal{C}_t^M = \{(m, e_t^M(m), a_{t+1}^M(m), v_t^M(m)) : m \in \mathcal{M}\},$$

where $a_{t+1}^M(m) = U(a_t, e_t^M(m))$ and $v_t^M(m) = \mathcal{V}(x, a_{t+1}^M(m))$. The selector for the first phase is

$$\hat{m}_t^M = \arg \max_{m \in \mathcal{M}} q_t^M(m),$$

with deterministic tie-breaking. The online artifact advances to $a_{t+1} = a_{t+1}^M(\hat{m}_t^M)$. Thus all branches are verified for diagnostics, but only the top-probability branch is visible to the model in the next online state.

Definition 6 (Verified trajectory tensor). A run produces both an online trajectory

$$\tau^{\text{on}} = ((h_t, q_t^M, \hat{m}_t^M, e_t^M(\hat{m}_t^M), v_t^M(\hat{m}_t^M), b_{t+1}))_{t=0}^{T-1}$$

and a verified tensor

$$\mathcal{Z} = \{(t, m, e_t^M(m), v_t^M(m), \mathbf{1}[m = \hat{m}_t^M]) : t < T, m \in \mathcal{M}\}.$$

Assumption 1 (Verifier objectivity). For fixed x , a , and random seed, $c(x, a)$ is exact and $\mathcal{L}(x, a)$ is reproducible up to bounded measurement noise. Incorrect artifacts are distinguishable from correct artifacts.

Assumption 2 (Mode coverage). At each diagnostic step the protocol elicits one valid candidate edit for each primary mode. If generation fails for a mode, the candidate is recorded as invalid and receives an incorrect verifier outcome.

Assumption 3 (Fixed taxonomy). The primary mode classifier is fixed before holdout evaluation. It may combine model-declared mode and diff-inferred mode, but the primary analysis uses the same deterministic rule for all backbones.

Assumption 4 (Fair comparison). Backbones are compared on identical workload profiles, budgets, prompts, mode definitions, safety gates, and verifier calls. Only the model identity changes within a comparison cell.

Assumption 5 (Teacher separation). Closed-source teacher models may generate labels or patches for training data, but only open-weight student models are post-trained. Evaluation of post-trained students is performed on held-out profiles and fresh runs.

4 Methodology

4.1 Gains and productive-mode proxies

Let $L_t = \mathcal{L}(x, a_t)$ be the loss of the current online artifact. For a candidate edit of mode m , define its verified gain

$$g_t^M(m) = L_t - \mathcal{L}(x, U(a_t, e_t^M(m))).$$

If the candidate is incorrect, the main protocol assigns a fixed negative penalty $-\lambda$ after loss normalization. A robustness analysis instead uses correctness gating, setting $[g_t^M(m)]_+ = 0$ for incorrect candidates.

The productive-mode proxy is a distribution over modes built from verified gains:

$$p_t^{\star, M}(m) = \frac{[g_t^M(m)]_+}{\sum_{m' \in \mathcal{M}} [g_t^M(m')]_+},$$

when at least one positive gain exists. If all gains are non-positive, the main analysis uses a uniform fallback and a secondary analysis uses the least-bad mode as a hard target.

4.2 Routing mismatch

Routing is the model’s allocation of probability mass over intervention modes before observing the current step’s verifier outcomes. We estimate mismatch in two complementary ways.

The distributional estimator is

$$\hat{\epsilon}_{\text{JSD}}(M) = \mathbb{E}_{t,r,x} \left[\text{JSD}(q_t^M, p_t^{\star,M}) \right],$$

where expectation is over steps t , runs r , and profiles x . The regret estimator is

$$R_t^M = \max_{m \in \mathcal{M}} g_t^M(m) - g_t^M(\hat{m}_t^M), \quad \hat{\epsilon}_{\text{regret}}(M) = \mathbb{E}[R_t^M].$$

The regret estimator is primary because it is measured in the same units as verified improvement; JSD is secondary because it captures distributional calibration.

4.3 Within-mode competence

Within-mode competence asks whether a model can implement a good patch once the intervention mode is fixed. We keep two estimands.

Runtime mode-conditioned competence. The mode-conditioned runtime estimate is

$$\hat{\phi}_{\text{runtime}}(M, m) = \mathbb{E}_{t,r,x} [g_t^M(m)], \quad \hat{\phi}_{\text{runtime}}(M) = \mathbb{E}_{m \sim \mu} [\hat{\phi}_{\text{runtime}}(M, m)],$$

where μ is either uniform over modes or the empirical productive-mode distribution. This quantity is available from the mode-complete tensor, but it is conditioned on states visited by the model’s own online trajectory.

Shared-checkpoint controlled competence. For a cleaner estimate, sample a checkpoint $\bar{h} = (x, \bar{a}, \bar{\mathcal{H}}, \bar{b})$ from a held-out pool and force each backbone to generate an edit for the same mode m . The controlled estimate is

$$\hat{\phi}_{\text{ctrl}}(M, m) = \mathbb{E}_{\bar{h}} [\mathcal{L}(x, \bar{a}) - \mathcal{L}(x, U(\bar{a}, e_M(\bar{h}, m)))].$$

The relative competence against a teacher or reference model M_0 is

$$\hat{\phi}_{\text{rel}}(M, m) = \hat{\phi}_{\text{ctrl}}(M, m) - \hat{\phi}_{\text{ctrl}}(M_0, m).$$

This is the preferred estimator when the goal is to separate local implementation quality from routing.

4.4 Feedback use

Feedback use is estimated in a separate diagnostic visibility condition. At a checkpoint, obtain q_t^{pre} , generate and verify the mode-complete candidate batch, expose either the selected result or all results, and query the model for q_t^{post} . The entropy-drop estimator is

$$\Delta H_t^M = H(q_t^{\text{pre}}) - H(q_t^{\text{post}}).$$

The alignment-gain estimator is

$$\Delta A_t^M = \text{JSD}(q_t^{\text{pre}}, p_t^{\star,M}) - \text{JSD}(q_t^{\text{post}}, p_t^{\star,M}), \quad \hat{G}(M) = \mathbb{E}[\Delta A_t^M].$$

Alignment gain is primary because a model can become more confident in the wrong direction; entropy alone does not indicate correctness.

4.5 Cost per step and endpoint gap

Let C_t^M be the cost of producing q_t^M and six edits. Cost can be measured in wall-clock time, tokens, dollars, or a normalized weighted combination. The primary cost estimator is

$$\hat{\kappa}(M) = \text{median}_{t,r,x} C_t^M.$$

Endpoint performance is measured by best verified loss within budget and success above a threshold η :

$$S_B^M = \min_{t \leq B} \mathcal{L}(x, a_t^M), \quad \text{Succ}_B^M(\eta) = \mathbf{1}[S_B^M \leq \eta].$$

A normalized performance gap relative to reference model M_0 is

$$\Delta_M = \mathbb{E}[S_B^{M_0} - S_B^M],$$

with sign convention adjusted so positive values favor M_0 if using loss reduction.

4.6 Diagnostic decomposition

The working decomposition is

$$\Delta_M \approx \log \frac{\kappa(M_0)}{\kappa(M)} + \phi(M) + G(M) - \epsilon(M).$$

We do not assume this expression is an exact identity over raw endpoint performance. Instead, it is a structured hypothesis: cost, local competence, feedback use, and routing mismatch explain distinct components of the performance gap. The following pathwise identity motivates the routing term.

Lemma 1 (Pathwise routing-regret accounting). *For any step t with mode-complete verified gains $\{g_t(m)\}_{m \in \mathcal{M}}$ and selected mode $\hat{m}_t$, define $g_t^* = \max_m g_t(m)$ and $R_t = g_t^* - g_t(\hat{m}_t)$. Then*

$$\sum_{t=0}^{T-1} g_t(\hat{m}_t) = \sum_{t=0}^{T-1} g_t^* - \sum_{t=0}^{T-1} R_t.$$

Proof. By definition, $g_t(\hat{m}_t) = g_t^* - R_t$ at each step. Summing over t proves the statement. $\square$

This identity says that, relative to the best mode already proposed at each checkpoint, the selected online trajectory loses exactly cumulative routing regret. It does not claim that the best proposed mode is globally optimal; it is a counterfactual lower-level accounting within the generated candidate set.

Lemma 2 (Distributional mismatch bound). *Assume gains lie in $[0, G_{\max}]$ and let $q, p \in \Delta(\mathcal{M})$. Then*

$$\left| \sum_m q(m)g(m) - \sum_m p(m)g(m) \right| \leq G_{\max} \text{TV}(q, p).$$

If JSD uses natural logarithms, $\text{TV}(q, p) \leq \sqrt{2 \text{JSD}(q, p)}$.

Proof. The first inequality follows from the definition of total variation and bounded gains. The second is Pinsker's inequality applied to the Jensen-Shannon divergence representation. $\square$

Proposition 1 (Identifiability of controlled within-mode competence). *Under Assumptions 1, 3, and 4, $\hat{\phi}_{\text{ctrl}}(M, m)$ identifies the expected verified gain of backbone M when the checkpoint and mode are fixed by the experimental design.*

Proof sketch. The checkpoint distribution and mode are externally fixed, so variation due to mode selection is removed. The only remaining model-dependent variable is the generated edit. The verifier maps the resulting artifact to correctness and loss under the same workload profile and budget-independent evaluation procedure. Averaging verified gains over sampled checkpoints estimates the desired conditional expectation. $\square$

Theorem 1 (Falsifiable diagnostic rule). *Suppose a backbone M is evaluated on held-out profiles and has a dominant standardized deficit in one component among ϵ , G , and ϕ . A post-training intervention targeted to that component yields a falsifiable prediction: improvement should be largest on the corresponding diagnostic estimator and should transfer to endpoint performance more than an untargeted fine-tuning baseline matched for data size and compute.*

Proof sketch. The theorem is a design-level implication rather than a distribution-free performance guarantee. The protocol produces separate datasets whose supervised labels target one component: routing labels for ϵ , feedback-to-next-action labels for G , and mode-conditioned patch labels for ϕ . If the diagnostic is valid and the intervention is effective, the targeted estimator should move in the predicted direction under held-out evaluation. Equal-data generic fine-tuning controls for training exposure. Failure of targeted improvement falsifies either the diagnostic, the intervention, or the assumed decomposition. $\square$

Table 1: Planned profile split. Names are placeholders to be bound to repository configurations before execution.

Split	Profile role	Purpose
Dev-1	small balanced	prompt and logging sanity checks
Dev-2	hard mixed	diagnostic estimator development
Dev-3	mode-stress	threshold selection and ablations
Holdout-1	balanced unseen seed	final model comparison
Holdout-2	distribution-shift unseen seed	robustness of diagnostics

4.7 Decision records for post-training

A decision record is the atomic training unit. For routing, the record contains h_t , q_t , verified gains by mode, the productive-mode proxy p_t^* , and the target mode distribution. For within-mode patching, the record contains h_t , fixed mode m , and a verified target patch. For feedback use, the record contains pre-feedback state, verifier batch results, and a target updated mode distribution or next action. The first post-training phase uses only routing records, because their labels are verifier-derived and do not require teacher patches.

5 Experimental Design

No results are reported in this draft. This section specifies the experimental protocol.

5.1 Task environment

The environment is the open-ended stateful query-engine task. Each run starts from a `baseline solution.py` implementing a naive `CandidateQueryEngine`. The required methods are `put`, `delete`, `get`, `range_sum`, `aggregate_count`, and `top_k`. A safety gate restricts imports and unsafe operations. The verifier evaluates exact output equivalence against a trusted reference engine and computes per-family latency and memory ratios. The scalar loss is the mean over workload families of a weighted latency-memory ratio.

5.2 Profiles and splits

A profile is a named workload-generator configuration: seed, initial size, key space, trace length, traces per family, operation families, repetitions, warmup prefix, and value range. We use three development profiles and two held-out profiles. Development profiles are used to finalize prompts, logging, mode classifier rules, loss normalization, and thresholds. Held-out profiles are used only for final evaluation of diagnostics and post-training.

5.3 Backbones

We compare three tiers under a shared single-agent scaffold: a strong teacher model, a mid-tier commercial model, and a weak open-weight code-capable student. The planned choices are Claude Opus as strong teacher, Claude Haiku as mid tier, and a Qwen code model as the post-trainable student, with exact model identifiers specified in configuration files at execution time [Anthropic, 2025, Qwen Team, 2026]. Closed-source backbones are never post-trained; they generate trajectories, targets, and upper bounds. The open-weight student is fine-tuned using LoRA or QLoRA.

5.4 Agent protocol

At each step, the agent must emit a strict JSON object with `mode_probs`, `mode_ranking`, and one edit proposal per primary mode. Each proposal is materialized in an isolated branch workspace, validated by the safety gate, and verified. The top-probability branch becomes the next online artifact. The full branch batch is logged but hidden from the agent in the first phase.

5.5 Metrics

Primary endpoint metrics are best verified loss within budget and success above a threshold within budget. Secondary diagnostics are routing regret, JSD to productive-mode proxy, runtime mode-conditioned competence, shared-checkpoint

controlled competence, feedback alignment gain, entropy drop, cost per step, invalid-patch rate, and verifier-call cost.

5.6 Experiments

E1: Instrumentation sanity. Run each backbone on the smallest development profile for a small budget. Verify that all six branches are generated, safety-gated, verified, and logged. Check that mode probabilities sum to one and that inferred modes agree with requested modes above a pre-specified threshold.

E2: Endpoint comparison. Evaluate the three backbones under identical budgets on all dev and holdout profiles. Test whether endpoint rankings differ from diagnostic rankings.

E3: Routing diagnostics. Compute ϵ_{regret} and ϵ_{JSD} from the verified branch tensors. Test whether weaker models allocate more mass to unproductive modes and incur greater routing regret.

E4: Within-mode competence. Estimate ϕ_{runtime} from branch tensors and ϕ_{ctrl} from shared checkpoints. Test whether a weak model’s deficit remains after mode is fixed.

E5: Feedback-use diagnostic. On a checkpoint sample, run both limited-visibility and full-feedback conditions. Query each model for pre- and post-feedback mode distributions. Estimate alignment gain G . Test whether full counterfactual feedback disproportionately helps models with better verifier interpretation.

E6: Routing post-training. Build a routing dataset from verifier-derived productive-mode proxies on development profiles. Fine-tune the open-weight student with LoRA/QLoRA to emit calibrated mode probabilities. Evaluate on held-out profiles. Compare against: no fine-tuning, generic trajectory imitation with matched records, random-label control, and teacher-distribution distillation if available.

E7: Ablations. Ablate primary vs secondary mode labels, hard vs soft productive-mode targets, top-1 visibility vs full visibility, full history vs compressed history, and fixed $K = 6$ vs adaptive candidate generation.

5.7 Hyperparameters

Planned ranges are: steps per run $T \in \{10, 20, 40\}$, candidate branches $K = 6$ for main experiments, model temperature $\in \{0.2, 0.5, 0.8\}$, branch timeout $\in [120, 300]$ seconds, LoRA rank $\in \{8, 16, 32\}$, learning rate $\in \{1e-5, 3e-5, 1e-4\}$, and routing target type $\in \{\text{soft}, \text{hard}\}$. All hyperparameters are chosen on development profiles only.

6 Discussion

Success modes. A strong outcome would show that endpoint performance gaps decompose into stable diagnostic patterns. For example, the weak student may assign high probability to `caching` when `indexing` or `summaries` are consistently more productive, yielding high routing regret. If routing post-training then improves held-out success more than generic fine-tuning, the work supports the claim that trajectories can guide targeted post-training.

A second success mode is mechanistic separation. A model may exhibit good routing but poor controlled within-mode competence. This would imply that mode-choice supervision is insufficient and motivate patch-level distillation. Another model may generate competent patches but fail to update after verifier feedback; this would motivate feedback-use training.

Failure modes. If mode taxonomy is unstable, routing estimates may be noisy. If most edits are compound, primary-mode labels may hide important causal structure. If all modes are unproductive late in a run, productive-mode proxies become weak and should be interpreted as local rather than global optimality signals. If endpoint improvements do not follow diagnostic improvements, the decomposition may be missing interaction terms, such as context management, prompt following, or search diversity.

Ablation directions. The most important ablations are mode granularity, visibility regime, state history, and target construction. We expect coarse modes to improve statistical power but reduce mechanism specificity; fine modes to improve interpretability but reduce reuse; and fixed $K = 6$ to be diagnostically clean but less realistic than adaptive generation.

Limitations. This draft specifies an experimental design rather than completed empirical results. The formal decomposition is a structured hypothesis with exact sub-identities, not a theorem that endpoint gaps must always decompose additively. The task is intentionally narrower than full software engineering; its value lies in instrumentation and causal diagnosis.

7 Future Directions

Phase 1: Canonical instrumentation. Implement the mode-complete branch generator, verifier logging, mode classifier, productive-mode proxy, and estimator suite. Validate on development profiles.

Phase 2: Backbone diagnostics. Run strong, mid, and weak backbones under identical budgets. Estimate endpoint metrics and diagnostic quantities. Select the dominant weak-backbone deficit before any post-training.

Phase 3: Routing post-training. Train the open-weight student on verifier-derived routing records. Evaluate on held-out profiles against no-training and generic-training controls.

Phase 4: Feedback-use training. Introduce full-feedback diagnostic episodes and construct verifier-summary-to-next-action records. Test whether feedback-use training improves alignment gain and endpoint performance.

Phase 5: Within-mode patch distillation. Use strong-model or verifier-selected patches to train mode-conditioned editing. Evaluate on shared checkpoints and full runs.

Phase 6: Generalization across tasks. Apply the same formalism to other verifiable artifact-optimization domains: query planners, training scripts, proof states, workflow graphs, or repository-level bug fixing.

8 Conclusion

We proposed verifiable agentic optimization as a task-first framework for studying language-model agents that iteratively modify artifacts under verifier feedback and budget constraints. The central claim is that such tasks should not be reduced to endpoint scores. By forcing a distribution over semantic intervention modes and verifying one candidate edit per mode, a run yields a rich trajectory tensor that exposes routing mismatch, feedback use, within-mode competence, and cost. These diagnostics can then drive targeted post-training, beginning with routing supervision for an open-weight student. The broader significance is methodological: a benchmark can be more than a race. If instrumented correctly, it becomes a diagnostic device and a source of supervision for improving the models it evaluates.

References

- Anthropic. Claude model documentation and claude code documentation. <https://platform.claude.com/docs/> and <https://code.claude.com/docs/>, 2025.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, 2023.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps. In *NeurIPS Datasets and Benchmarks*, 2021.

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378 (6624):1092–1097, 2022.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kai Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, 2024.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.
- Qwen Team. Qwen3-coder-next model card. <https://huggingface.co/Qwen/Qwen3-Coder-Next>, 2026.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023b.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.